

Entregable 4. Desarrollo funcional, diseño de software y versionamiento en Git

Piloto funcional con frontend en Angular y backend en .NET, versionado en Git. Este documento sirve de guía: donde está el código, cómo levantarlo y qué revisar.

4.1 Repositorio

<https://github.com/asinay11/sigop>

Las instrucciones completas de compilación, ejecución, configuración, dependencias, migraciones y datos de prueba están en el archivo README del repositorio.

4.2 Qué implementa el piloto

Requisito del enunciado	Estado
Registro de solicitudes	Implementado
Consulta de solicitudes e historial	Implementado
Actualización de una solicitud	Implementado, solo en estado Registrada
Cambio de estado	Implementado, validado por la máquina de estados
Validaciones de negocio	Implementado, en la capa de dominio
Persistencia	PostgreSQL 17 con Entity Framework Core 10
Autenticación y autorización	JWT y segregación por entidad
Manejo global de errores	Implementado, con códigos HTTP estables
Publicación de al menos un evento en RabbitMQ	Implementado: dos tipos de evento
Organización por capas	Seis proyectos, dependencias hacia adentro
Pruebas automatizadas básicas	Pendiente
README e instrucciones de ejecución	Implementado
Archivo .gitignore	Implementado
Ramas de trabajo	master, develop y feature por área
Pull request	De develop a master

4.3 Organizacion por capas

```
backend/  
  Sigop.Dominio          agregado, maquina de estados, eventos  
  Sigop.Aplicacion       casos de uso, puertos y DTOs  
  Sigop.Infraestructura  EF Core, RabbitMQ, seguridad, integraciones  
  Sigop.Api              controladores REST y legado SOAP simulado  
  Sigop.Worker.Outbox    despachador del outbox  
  Sigop.Worker.Consumidor consumidor orquestador  
database/               esquema y datos de prueba  
frontend/sigop-portal/  portal Angular
```

Sigop.Dominio no tiene ni un solo paquete NuGet ni referencia a otro proyecto: las dependencias apuntan hacia adentro y la infraestructura implementa los puertos que declara la capa de aplicacion. Es lo que permite probar las reglas de negocio sin base de datos ni broker.

4.4 Como levantarlo

Requiere .NET 10, Node 20, PostgreSQL 17 y RabbitMQ. Crear la base y cargar los datos de prueba:

```
psql -U postgres -c "CREATE DATABASE sigop;"  
psql -U postgres -d sigop -f database/01-esquema.sql  
psql -U postgres -d sigop -f database/02-datos-prueba.sql
```

Los tres archivos appsettings.json del repositorio son plantillas con marcadores; el README trae el bloque que genera la configuracion local. Despues, tres procesos en terminales separadas y el portal:

```
dotnet run --project backend/Sigop.Api          (puerto 5005)  
dotnet run --project backend/Sigop.Worker.Outbox  
dotnet run --project backend/Sigop.Worker.Consumidor
```

```
cd frontend/sigop-portal && npm install && npm start    (puerto 4200)
```

4.5 Usuarios de prueba

Ambos con contrasena 12345. Cada uno ve solo las solicitudes de su entidad.

Usuario	Nombre	Entidad
jperez	Juan Perez	11130007 - Ministerio de Finanzas Publicas
rmorales	Rosa Morales	11130008 - Ministerio de Gobernacion

Entrar con uno y luego con el otro demuestra la segregacion multientidad: cambian las solicitudes listadas, las unidades ejecutoras disponibles y el nombre de la entidad en la barra superior.

4.6 Escenarios de demostracion

El resultado depende del monto, de forma determinista.

Monto	Legado	Banco	Estado final
Menor a Q500,000	Disponible	Confirma	Ejecutada
Entre Q500,000 y Q1,000,000	Disponible	Agota el tiempo	Fallida, reprocesable
Mayor a Q1,000,000	Sin disponibilidad	No se invoca	Rechazada

Una solicitud fallida se reprocesa desde el detalle, hasta 3 veces. Para demostrar la escalera de reintentos y la cola de mensajes muertos basta con detener la API mientras el consumidor sigue arriba: el legado simulado vive dentro de la API, de modo que la llamada falla y entra el mecanismo de reintentos.

4.7 Resultados verificados en ejecucion

Comprobacion	Resultado
Flujo completo hasta Ejecutada	Menos de 4 segundos
Rechazo por falta de disponibilidad	Menos de 4 segundos
Fallo bancario y reproceso	Menos de 10 segundos
Escalera de reintentos con el legado caido	Fallida y DLQ a los 110 segundos
Outbox	13 mensajes, todos publicados
Idempotencia	13 marcas, una por mensaje, sin duplicados
Aislamiento entre entidades	Un usuario de otra entidad recibe HTTP 404
Peticion sin token	HTTP 401
Limite de intentos de autenticacion	HTTP 429 a partir del intento 11

4.8 Versionamiento

Estrategia de ramas: master siempre desplegable, develop de integracion y una rama feature por area de trabajo. Las fusiones a develop se hacen con --no-ff para que queden visibles en el grafo, y de develop a master mediante pull request.

Rama	Contenido
feature/modelo-db	Esquema y datos de prueba
feature/backend-dominio	Agregado, estados y maquina de estados
feature/backend-aplicacion	Casos de uso, puertos y DTOs
feature/backend-infraestructura	Persistencia, outbox, seguridad, mensajería y consumidor
feature/api-rest	Controladores y arranque de la API
feature/workers	Despachador y consumidor
feature/portal-angular	Portal

Rama	Contenido
feature/ci	Integracion continua
feature/documentacion	README y documentos de arquitectura

Los commits son pequenos y descriptivos, y representan avances reales: cada uno anade un archivo o un area concreta, con un mensaje que dice que aporta. La evidencia de integracion es el pull request de develop a master.

4.9 Integracion continua

El archivo .github/workflows/ci.yml ejecuta tres trabajos en cada push y en cada pull request: compilacion del backend con pruebas y revision de dependencias vulnerables, compilacion del portal, y una comprobacion que falla si se versiona configuracion local con secretos o si una plantilla trae un valor real en lugar de un marcador.

4.10 Pendiente

Las pruebas automatizadas no estan implementadas. La prioridad seria el dominio, que es lo mas barato de probar y donde vive el riesgo: la maquina de estados, verificando cada transicion legal y el rechazo de las ilegales. Despues la capa de aplicacion, para la segregacion por entidad y el reproceso, y por ultimo integracion para el outbox y la idempotencia.